

# Module 5 — Multithreading & Concurrency

High priority for backend roles. Product companies dig into `volatile`, the memory model, and deadlock; service companies ask lifecycle, `synchronized`, and `ExecutorService`.

**Node.js bridge:** Node is single-threaded with an event loop; you offload with worker threads or async IO. Java uses **real OS threads** with shared memory — which means you personally own **visibility** and **atomicity** correctness. This is the biggest mental shift from Node.

## 5.1 Process vs Thread · Thread Lifecycle

### Process vs Thread

- **Process** — own memory/address space; heavy; isolated.
- **Thread** — lightweight unit inside a process; **shares heap/metaspaces**, has its own **stack** + PC. Cheap to create, but shared mutable state needs synchronization.

### Thread Lifecycle (states in `Thread.State`)

```
NEW --start()-> RUNNABLE --scheduler--> (running)
RUNNABLE <-> BLOCKED (waiting for monitor lock)
RUNNABLE <-> WAITING (wait()/join()/park(), no timeout)
RUNNABLE <-> TIMED_WAITING (sleep(t)/wait(t))
--> TERMINATED (run() returns/throws)
```

- **RUNNABLE** covers both “ready” and “running” (JVM doesn’t distinguish).
- **BLOCKED** = waiting to acquire a monitor; **WAITING** = waiting for another thread’s signal.

## 5.2 Runnable · Callable · ExecutorService · ThreadPool

### Runnable vs Callable

|                    | Runnable                    | Callable                               |
|--------------------|-----------------------------|----------------------------------------|
| Method             | <code>void run()</code>     | <code>V call() throws Exception</code> |
| Returns            | nothing                     | a value                                |
| Checked exceptions | no                          | yes                                    |
| With executor      | <code>execute/submit</code> | <code>submit → Future&lt;V&gt;</code>  |

### ExecutorService & thread pools (never `new Thread()` in prod)

- Creating threads manually is unbounded → OOM under load. Use a **pool** to cap and reuse threads.
- **Executors** factories: `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newVirtualThreadPerTaskExecutor()` (Java 21).
- **Prefer constructing `ThreadPoolExecutor` directly** in production for a **bounded queue** and explicit rejection policy — `newFixedThreadPool` uses an *unbounded* `LinkedBlockingQueue` that can OOM.

## Internal Working — ThreadPoolExecutor

```
submit(task):
  if activeThreads < corePoolSize      -> create core thread, run
  else if queue not full                -> enqueue task
  else if activeThreads < maxPoolSize   -> create non-core thread
  else                                 -> RejectedExecutionHandler (Abort/CallerRuns/Discard)
idle non-core threads die after keepAliveTime
```

## Memory Diagram

```
[submit] -> core threads busy? -> [ bounded queue ] full? -> spawn up to max -> full? -> REJECT
                                           |
                                           RejectedExecutionHandler
```

## Coding Example

```
ExecutorService pool = new ThreadPoolExecutor(
    4, 8, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100),           // bounded queue (backpressure)
    new ThreadPoolExecutor.CallerRunsPolicy()); // graceful rejection

Future<Integer> f = pool.submit(() -> expensiveCompute()); // Callable
Integer result = f.get(2, TimeUnit.SECONDS);               // blocks with timeout
pool.shutdown();                                           // then awaitTermination
```

## Best Answer

"I never spawn raw threads; I use an `ExecutorService`. In production I build a `ThreadPoolExecutor` with core/max sizes, a **bounded** queue for backpressure, and an explicit rejection policy — because `Executors.newFixedThreadPool` hides an unbounded queue that can OOM. `Runnable` returns nothing; `Callable` returns a `Future`."

## 5.3 synchronized · volatile · Atomic — visibility & atomicity

### The Java Memory Model (JMM) — the concept behind everything

Each thread may cache variables in registers/CPU caches. Without synchronization, one thread's write may **never become visible** to another. The JMM defines **happens-before**: `synchronized` release → acquire and `volatile` write → read establish visibility ordering.

### synchronized

- Mutual exclusion + visibility. Acquires an object's **monitor** (intrinsic lock). Only one thread holds it; others **BLOCKED**.
- On method (`synchronized void m()`) locks `this` (or the Class for static); on block `synchronized(lock){}` locks a chosen object.
- Reentrant (same thread can re-acquire). Guarantees both **atomicity** of the block and **visibility** of all changes made inside.

### volatile

- Guarantees **visibility** and ordering (no caching, no reordering across it) — but **NOT atomicity** of compound ops (`count++` is read-modify-write → still racy even if `volatile`).
- Use for **flags** (`volatile boolean running`) and the double-checked-locking singleton reference.

### Atomic classes

- `AtomicInteger/AtomicLong/AtomicReference` provide **lock-free** atomic ops via **CAS** (Compare-And-Swap, a CPU instruction). `incrementAndGet()` is atomic without locking.

- Higher throughput than `synchronized` under contention for simple counters. `LongAdder` scales even better for hot counters.

## Memory Diagram

```
count++ (NOT atomic):
  T1 read 5 | T2 read 5 | T1 write 6 | T2 write 6  -> lost update!
Fixes: synchronized{count++} OR AtomicInteger.incrementAndGet() (CAS retry loop)
volatile boolean flag: write by T1 immediately visible to T2 (no cache staleness)
```

## Comparison

| Tool                       | Atomicity                   | Visibility | Cost            |
|----------------------------|-----------------------------|------------|-----------------|
| <code>volatile</code>      | No (single read/write only) | Yes        | cheap           |
| <code>synchronized</code>  | Yes                         | Yes        | lock (blocking) |
| <code>Atomic*</code> (CAS) | Yes (single var)            | Yes        | lock-free, fast |

## 5.4 Locks (`java.util.concurrent.locks`)

- **ReentrantLock** — explicit lock/unlock (always in `finally`), supports `tryLock(timeout)`, fairness, interruptible waits — more flexible than `synchronized`.
- **ReadWriteLock / StampedLock** — many readers OR one writer; great for read-heavy caches.
- Always: `lock.lock(); try { ... } finally { lock.unlock(); }`.

```
private final ReentrantLock lock = new ReentrantLock();
void safe() {
    lock.lock();
    try { /* critical section */ }
    finally { lock.unlock(); } // MUST unlock in finally
}
```

## 5.5 Deadlock & Race Condition

### Race Condition

Outcome depends on thread timing (e.g., lost update on `count++`, check-then-act on a map). Fix with synchronization, atomics, or immutable/confined state.

### Deadlock — the 4 Coffman conditions

Mutual exclusion + hold-and-wait + no preemption + **circular wait**. Break any one.

```
T1: lock A ... wait for B
T2: lock B ... wait for A    -> both stuck forever
```

**Prevention:** acquire locks in a **global consistent order**; use `tryLock` with timeout; minimize lock scope; prefer higher-level concurrency utilities.

### Best Answer

“A race condition is when correctness depends on timing — like two threads doing `count++` and losing an update; I fix it with atomics or synchronization. A deadlock needs all four Coffman conditions; the practical fix is a consistent global lock ordering or `tryLock` with a timeout so a thread backs off instead of waiting forever.”

## 5.6 CompletableFuture (basic)

Async composition without blocking — Java's answer to JS Promises.

```
CompletableFuture
    .supplyAsync(() -> fetchUser(id), pool)           // async, on a pool
    .thenApply(User::profile)                         // transform (map)
    .thenCompose(p -> fetchOrdersAsync(p))           // chain another future (flatMap)
    .thenCombine(fetchRecommendations(id), Result::merge) // join two futures
    .exceptionally(ex -> Result.empty())             // error handling
    .thenAccept(this::respond);                      // consume
```

- `thenApply` = map; `thenCompose` = flatMap; `thenCombine` = zip; `exceptionally/handle` = error handling.
- Non-blocking — don't call `.get()` on the hot path; compose instead.
- Always supply your own executor for blocking work (default is the common ForkJoinPool).

**Node bridge:** `supplyAsync` ≈ `new Promise`, `thenApply` ≈ `.then`, `exceptionally` ≈ `.catch`.

## Module 5 — Top 25 Interview Questions (senior answers)

1. **Process vs thread?** Own memory vs shared heap, own stack.
2. **Thread states?** NEW, RUNNABLE, BLOCKED, WAITING, TIMED\_WAITING, TERMINATED.
3. **Runnable vs Callable?** void vs returns value/throws checked → Future.
4. **Why ExecutorService over new Thread?** Reuse, bounded, backpressure, lifecycle.
5. **newFixedThreadPool risk?** Unbounded queue → OOM; build ThreadPoolExecutor with bounded queue.
6. **ThreadPoolExecutor params?** core, max, keepAlive, queue, rejection handler.
7. **Rejection policies?** Abort, CallerRuns, Discard, DiscardOldest.
8. **synchronized — what does it guarantee?** Atomicity + visibility via monitor.
9. **volatile — what it does/doesn't?** Visibility+ordering, NOT compound atomicity.
10. **Is `count++` atomic with volatile?** No — read-modify-write race.
11. **Atomic classes / CAS?** Lock-free atomic single-var ops.
12. **synchronized vs Lock?** Implicit vs explicit (tryLock, fairness, interruptible).
13. **What is reentrancy?** Same thread can re-acquire a held lock.
14. **wait/notify vs sleep?** Releases lock + needs monitor vs just pauses, keeps lock.
15. **Race condition?** Timing-dependent incorrect result.
16. **Deadlock & 4 conditions?** Mutual exclusion, hold-wait, no preemption, circular wait.
17. **Prevent deadlock?** Global lock ordering, tryLock timeout, minimal scope.
18. **Happens-before?** JMM ordering from locks/volatile that guarantees visibility.
19. **ThreadLocal use & risk?** Per-thread state; leak in pools if not `remove()`.
20. **CompletableFuture vs Future?** Composable/non-blocking vs blocking `get`.
21. **thenApply vs thenCompose?** map vs flatMap.
22. **ConcurrentHashMap vs synchronizedMap?** Bucket-level/CAS vs single lock.
23. **Producer-consumer?** `BlockingQueue` (put/take block).
24. **CountDownLatch vs CyclicBarrier vs Semaphore?** One-shot wait / reusable barrier / permits.
25. **Virtual threads (Java 21)?** Cheap JVM-scheduled threads for high-concurrency IO  
(`newVirtualThreadPerTaskExecutor`).

## Module 5 — Top Coding Questions

- Thread-safe counter (synchronized vs AtomicInteger vs LongAdder).

- Producer-consumer with `BlockingQueue`.
- Thread-safe singleton (double-checked locking with `volatile`; or enum/holder idiom).
- Print odd/even alternately with two threads (wait/notify or Lock+Condition).
- Run N tasks in parallel and combine results with `CompletableFuture`.
- Reproduce and then fix a deadlock.

## Module 5 — Common Follow-ups

- “Why is `volatile` not enough for a counter?”
- “Your fixed pool works in dev but OOMs in prod under a spike — why?”
- “How would you cap concurrency to 10 outbound calls?” (Semaphore / bounded pool.)

## Module 5 — One-Page Cheat Sheet

```
Process=own memory; Thread=shared heap+own stack
States: NEW/RUNNABLE/BLOCKED/WAITING/TIMED_WAITING/TERMINATED
Runnable(void) vs Callable(V, Future). Use ExecutorService, bounded queue, rejection policy
synchronized = atomicity+visibility (monitor, reentrant)
volatile = visibility only (NOT count++). Atomic/CAS = lock-free atomic single var
Lock: lock/unlock in finally; tryLock timeout; ReadWriteLock for read-heavy
Race = timing bug. Deadlock = 4 Coffman -> fix: global lock order / tryLock
CompletableFuture: supplyAsync/thenApply(map)/thenCompose(flatMap)/thenCombine/exceptionally
Singleton: enum or volatile double-checked locking
ThreadLocal: remove() in pools to avoid leaks. Java21 virtual threads for IO concurrency
```

## Module 5 — Mock Interview (answer, then continue)

1. “Two threads increment a shared `int` a million times each; the total is wrong. Explain why and give three fixes with trade-offs.”
2. “Design a thread pool config for a service that calls a slow downstream API — sizes, queue, rejection?”
3. “Explain `volatile` vs `synchronized` vs `AtomicInteger` and when you’d pick each.”
4. “Show me a deadlock and fix it two different ways.”
5. “Rewrite three blocking downstream calls to run concurrently and merge with `CompletableFuture`.”

*Continue to Module 6 when ready.*